Founding QA Engineer Take-Home Assessment

Application: FroggyTalk
Tester: Kyrillos Sedhom
Device: iPhone 11(iOS 26.3.1)

Part 1: Product Exploration (Using Live App)

Bug 1 – Back Navigation in onboarding flow not working

Title: Force Navigate to Third slide instead of easily going through onboarding screens

Steps to Reproduce

1) Open the app
2) Reach to the onboarding screen
3) Swipe to second or third slide
4) Try to go back to first slide

Expected Result

- Users should easily navigate through the slides., including returning to first slide.

Actual Result

- When attempting to navigate to the first slide, the app force redirects to the third slide.

Severity

- High- affect onboarding screen flow and usability which could lead to potential drop off during first time use.

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Bug 2 – Context mismatch because of Live Chat in Radio

Title: Live Chat is shared across different radio stations causing context mismatch

Prerequisites

Users logged in to access multiple radio stations within the app.

Steps to Reproduce

1. Open the app - Login
2. Choose the “Radio” icon from the bottom navigation bar.
3. Choose a Nigerian radio station
4. Open live chat and observe messages
5. Switch to another radio station
6. Observe live chat again

Expected Result

- Each radio station should have its own independent live chat session so conversations stay relevant to selected station

Actual Result

- Same live chat is displayed across different radio stations, causing users from different radio stations to see and interact in the same conversation thread.

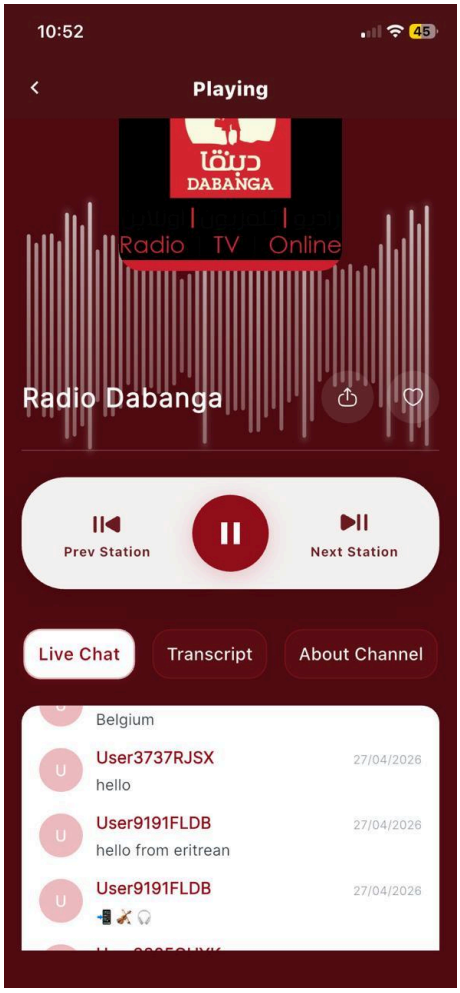
Severity

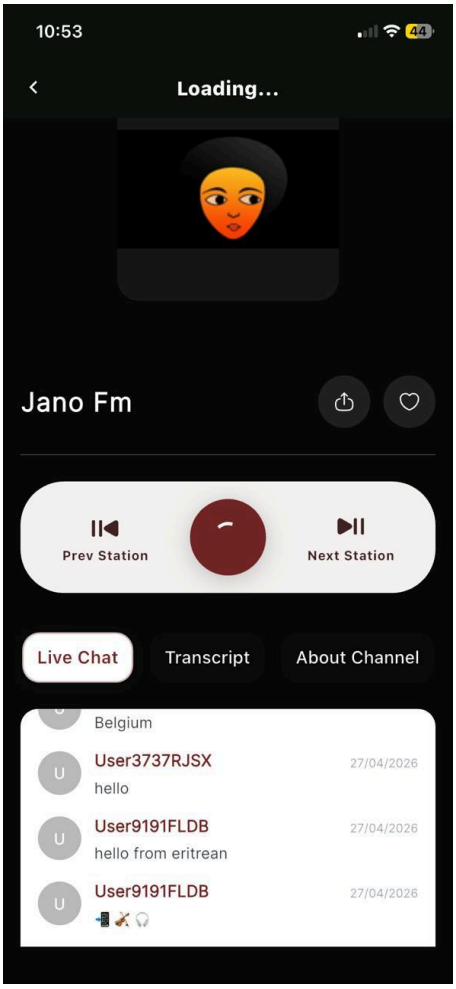
- Medium - Context mismatch reduces chat relevance and user experience.

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Screenshots





Bug 3 – Showing error for Contacts after Limited access to contacts

Title: After giving Permission to specific contacts, an error is shown in the home page

Prerequisites

1. Users logged in
2. Device has contacts available
3. User selects “Limited access to Selected contacts”

Steps to Reproduce

1. Open the app - Login
2. Choose “Contacts” button in home screen
3. Proceed to the contacts permission prompt, Choose “Limited Access” and choose specific contacts

4. Confirm Permission
5. Observe app behavior after permission granted

Expected Result

- App should accept Limited access without errors
- App sync selected contacts successfully
- Continue flow without interrupting user experience

Actual Result

- It shows an error message : Error: Contacts permission not granted

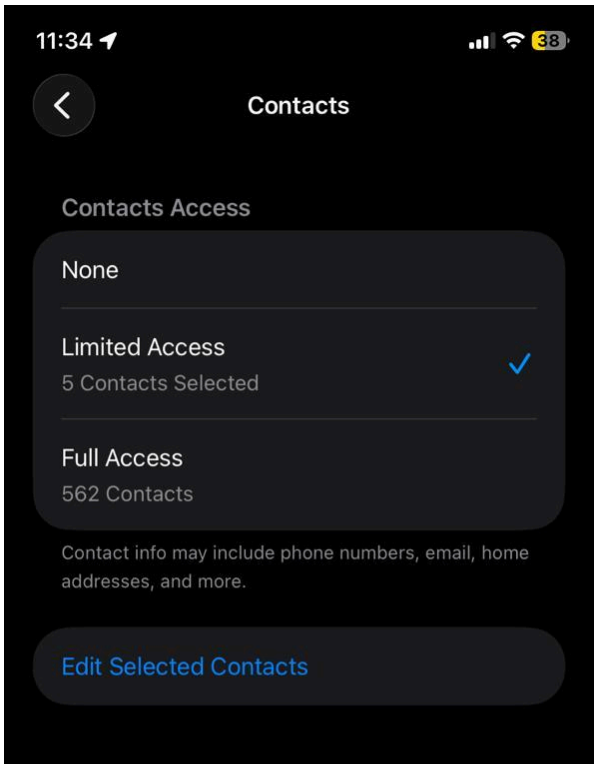
Severity

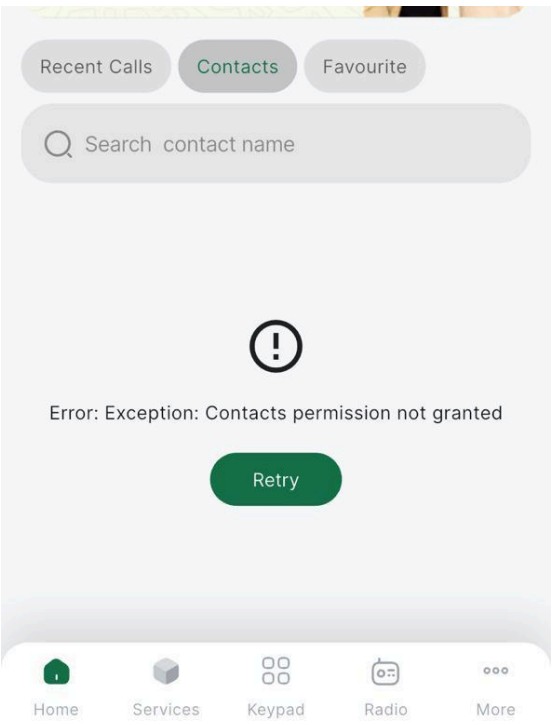
- High - Users not able to choose specific contacts to deal with and could lead to misunderstanding that the application is not working causing a potential dropoff .

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Screenshots





Bug 4 – Pay Utility Bills restricted to Nigeria without clear user Feedback

Title: Users are stuck on Nigeria when trying to choose country to Pay Utility Bills

Prerequisites

Users logged into FroggyTalk

Steps to Reproduce

1. Open the app - Login
2. Choose the “Services” icon from the bottom navigation bar.
3. Choose “Pay Utility Bills”
4. Press on Country dropdown to change it
5. Observe app behavior after pressing on country

Expected Result

- If the feature is intended to support Nigeria only: The UI should clearly indicate that the service is Nigeria-only.
- If multi-country support is intended: The country dropdown should open and Users are able to switch countries
- Proceed successfully with paying utility bills

Actual Result

- It shows only Nigeria and does not allow selection of any other country.
- No message or Indication explaining that this service is restricted to Nigeria only.

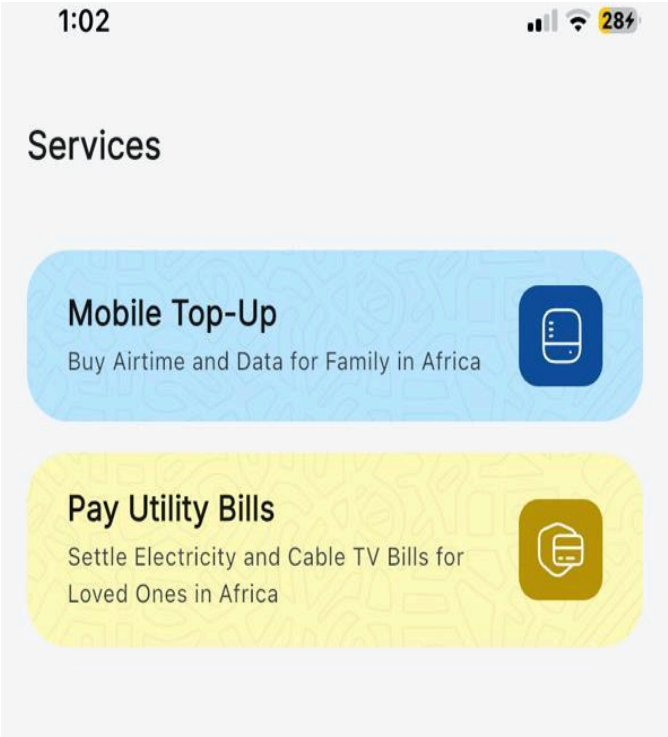
Severity

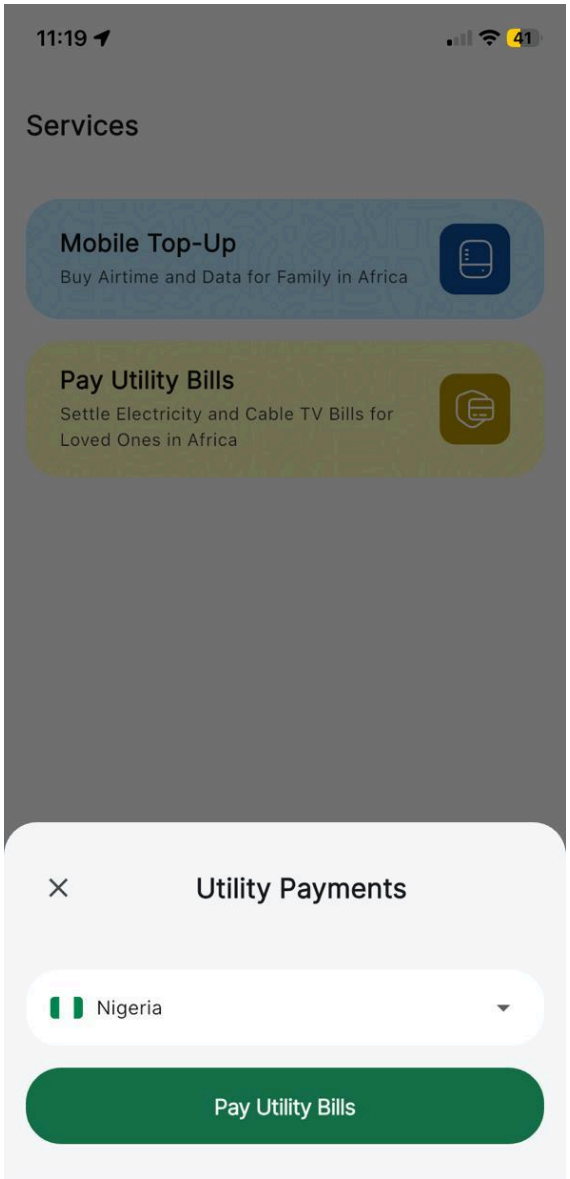
- Low to Medium - Affect user Understanding and experience (UX issue)

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Screenshots





Bug 5 – Buy Electricity screen Backend error on initial load

Title: Error message displayed on Buy Electricity screen with no user input

Prerequisites

Users logged into FroggyTalk

Steps to Reproduce

1. Open the app - Login
2. Choose the “Services” icon from the bottom navigation bar.
3. Choose “Pay Utility Bills”
4. Press on “Pay Utility Bills” under selected country
5. Tap on “Buy Electricity”
6. Choose Prepaid or Postpaid
7. Observe app behavior

Expected Result

- Screen should load normally without errors
- No Error message should appear before user interaction

Actual Result

- It shows a BackEnd Error message “No query results for Model [App\Models\BillExchange]”

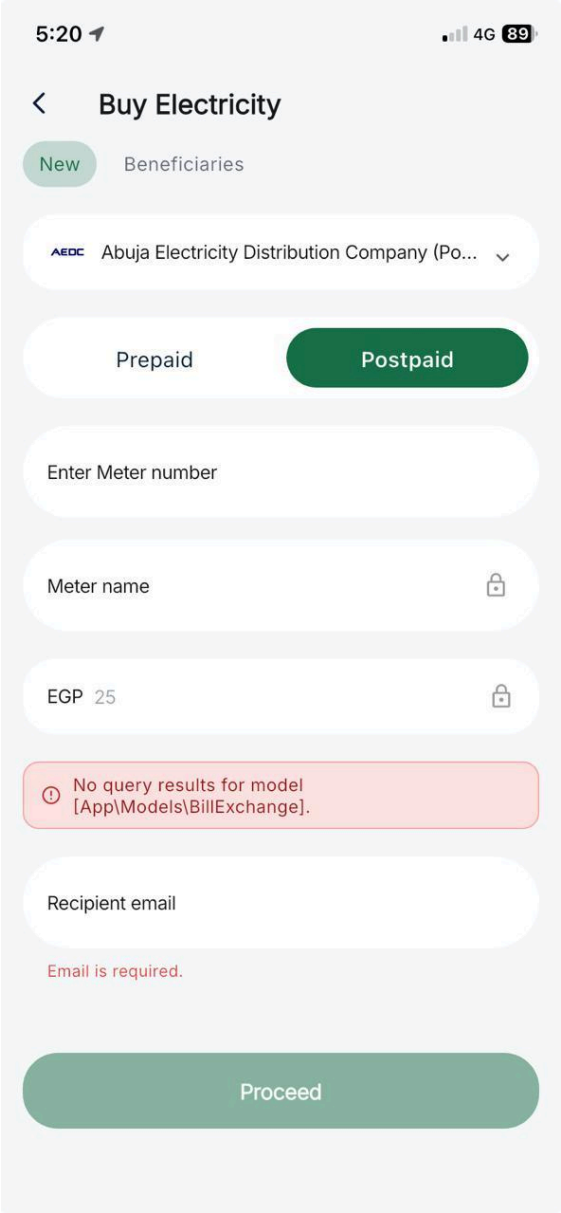
Severity

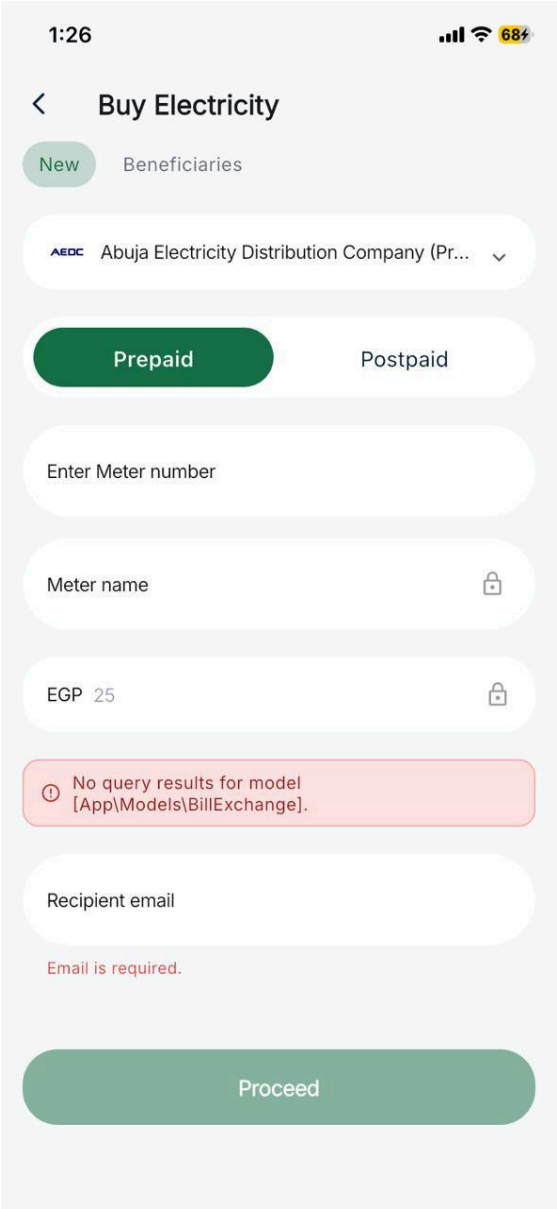
- High - Indicates failure in initial data loading and may confuse users before starting the flow

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Screenshots





Bug 6 – Mobile Top-Up stuck on loading for Airtime and Data bundles

Title: Airtime and Data bundle are stuck on loading state for Nigerian numbers in Mobile Top Up

Prerequisites

1. Users logged into FroggyTalk
2. The app has full permission to access contacts.
3. User has a nigerian number

Steps to Reproduce

1. Open the app - Login
2. Choose the “Services” icon from the bottom navigation bar.
3. Choose “Mobile Top up”
4. Type a nigerian number or search for one
5. Choose from “Airtime”and “Data Bundles”
6. Observe app behavior

Expected Result

- Screens should load normally without errors and should show data
- User should be able to proceed with the purchase

Actual Result

- It keeps loading with no results shown
- No data or error message is displayed
- User can’t proceed

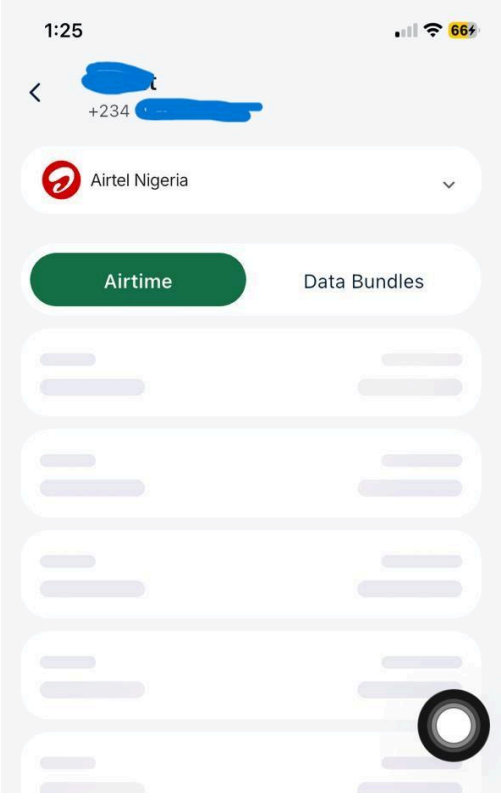
Severity

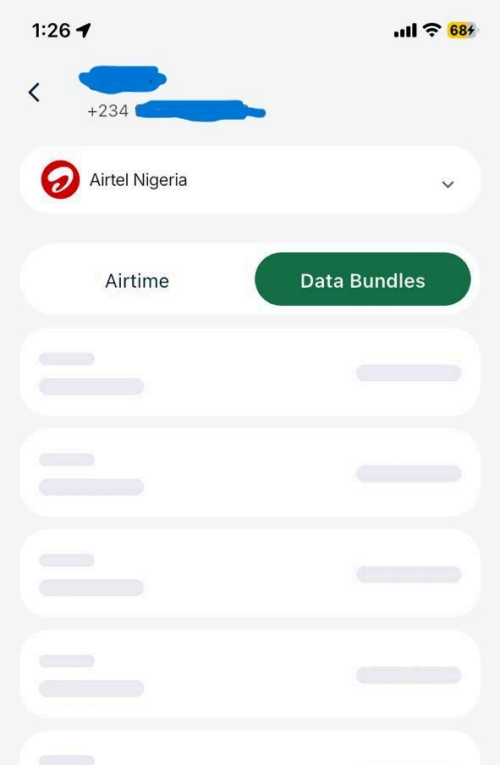
- High - Creates confusion for users and likely to create user drop off or lack of trust

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Screenshots





Bug 7 – Showing error in Mobile Top Up after Limited access to contacts

Title: Backend error is shown for recipient even having Limited access to contacts

Prerequisites

1. Users logged in
2. Device has contacts available
3. User selects “Limited access to Selected contacts”

Steps to Reproduce

1. Open the app - Login
2. Proceed to the contacts permission prompt, Choose “Limited Access” and choose specific contacts
3. Confirm Permission
4. Choose the “Services” icon from the bottom navigation bar.
5. Choose “Mobile Top up”
6. Observe app behavior

Expected Result

- App should accept Limited access without errors
- App sync selected contacts successfully
- Continue flow without interrupting user experience
- Screens should load normally without errors and should show selected contacts
- User should be able to proceed with the purchase if the number is valid

Actual Result

- It shows an error message : Error: Contacts permission not granted

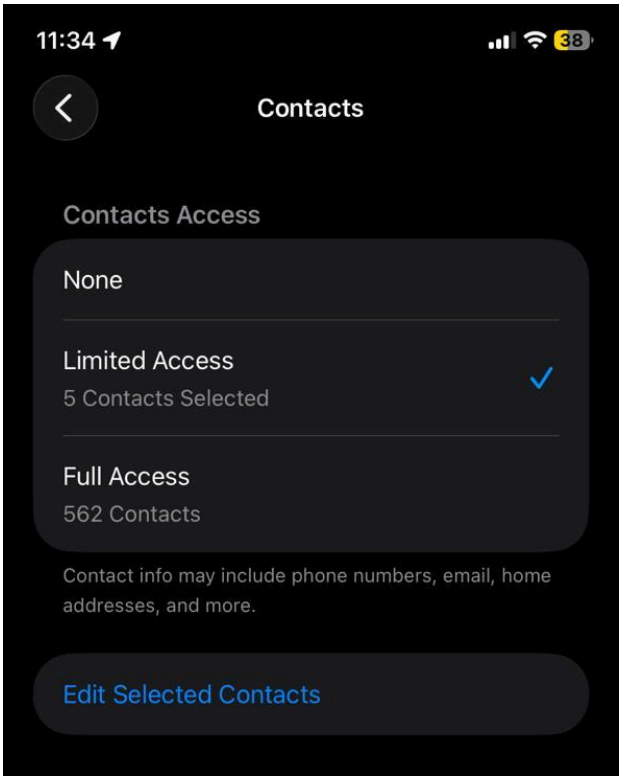
Severity

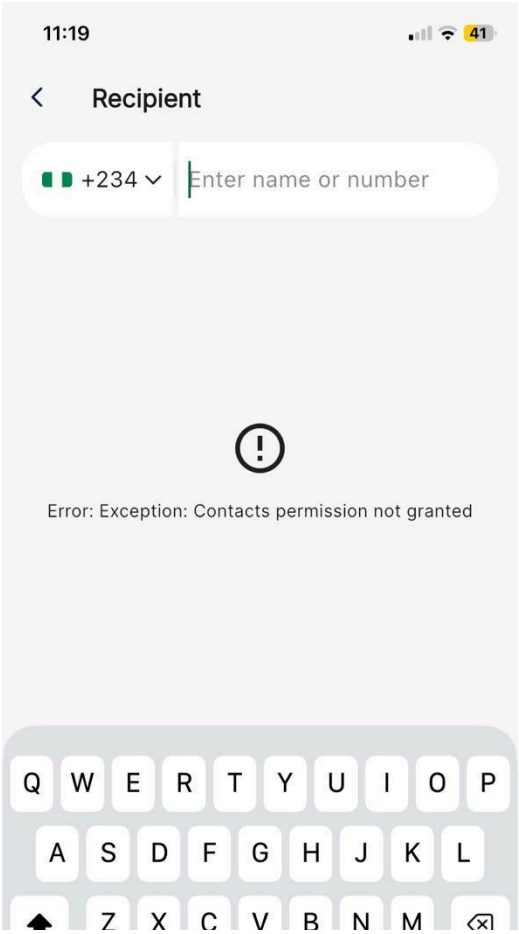
- High - Users not able to choose specific contacts to deal with and could lead to misunderstanding that the application is not working causing a potential dropoff .

Environment

- Device: iPhone 11
- OS: iOS 26.3.1

Screenshots





Top Risks to Product Quality as FroggyTalk Scales

- Live Chat Data Isolation Risk: Shared chat across radio stations indicates lack of proper session/channel isolation which can lead to misunderstanding and cross-user data leakage.
- Backend Reliability Issues: Errors like No query Results for Model and infinite loading suggest weak error handling and backend instability.
- Permission Handling Problem: Limited contacts access is not handled correctly which breaks core flows like top-up and contacts.
- Silent Failures / Poor Error Handling: Some flows show loading forever with no feedback which reduces trust and increases drop-off.
- Third-Party Dependency Risk: Services like mobile top-up and electricity payments may fail due to external APIs.

Part 2: QA Strategy & Ownership

1. QA Strategy (Manual vs Automation Balance)

In early-stage FroggyTalk, QA should focus on speed and risk coverage.

Manual Testing (70%):

- Exploratory testing of new features
- UX validation
- Edge cases and real user behavior simulation

Automation Testing (30%):

- Critical regression flows
- API validation
- Core user journeys (authentication, radio streaming, chat)

2. Risk-Based Testing Approach

I would prioritize testing based on user impact:

High Risk Areas:

- Authentication (OTP login)
- Mobile top-up / Utility payments
- Contacts permissions
- Live chat correctness and isolation
- Buying Credit

Medium Risk Areas:

- Profile setup flow
- Notifications
- UI consistency issues

Low Risk Areas:

- Minor UI issues
- Non-critical settings screens

3. CI/CD Integration Strategy

- Run smoke tests on every pull request
- Run full regression suite before staging deployment
- Block production release on critical test failures
- Integrate API + UI automation into pipeline

4. Release Confidence / Gating

A release is considered safe when:

- All critical test cases pass
- No open High severity bugs
- Core flows validated in staging
- No regression in authentication or streaming

5. Key Metrics to Track

- Test pass/fail rate
- Production defect leakage (escaped bugs)
- API failure rate
- Mean Time to Detect (MTTD)
- Coverage of critical flows

Part 3: Automation Approach

1. Tooling Choices

- Python + Pytest → test structure
- Requests → API testing
- Appium (XCUITest) → iOS UI automation

2. What I Chose to Automate (and Why)

I focused on high-risk, high-value flows :

- Mobile Top-Up (Airtime Purchase Flow)

3. API Test Example

```
import requests


BASE_URL = "https://example.com/api"


def test_mobile_topup_products():
    payload = {
        "phone": "+23408012345678",
```

```
        "country": "NG"
    }

    response = requests.post(f"{BASE_URL}/mobile-topup/options", json=payload)

    assert response.status_code == 200
    data = response.json()

    assert "airtime" in data
    assert "data_bundles" in data
    assert isinstance(data["airtime"], list)
```

4. iOS UI Automation (Appium Example)

```
from appium import webdriver

def test_mobile_topup_flow():
    caps = {
        "platformName": "iOS",
        "deviceName": "iPhone 11",
        "automationName": "XCUITest",
        "app": "/path/to/froggytalk.app"
    }

    driver = webdriver.Remote("http://localhost:4723/wd/hub", caps)

    # Navigate to Services
    driver.find_element("accessibility id", "ServicesTab").click()
```

```
# Open Mobile Top-Up
driver.find_element("accessibility id", "MobileTopUp").click()

# Enter phone number
driver.find_element("accessibility id", "PhoneInput").send_keys("08012345678")

# Select Airtime
driver.find_element("accessibility id", "AirtimeOption").click()

# Verify next screen loads
assert driver.find_element("accessibility id", "PaymentButton").is_displayed()

driver.quit()
```

README:

Setup:

- Download and Install Python 3.x
- After installing python open command prompt to Install dependencies:
  pip install pytest requests Appium-Python-Client

Navigate to the directory in which your API Tests exist then Run:

pytest test_api.py

For Running Mobile Tests:

- Start Appium server: appium
- Run: pytest test_ui.py

GitHub Repo:

https://github.com/Kyrillosadel/FroggyTalk-QA-Assessment/tree/main

5. Notes on Execution

This automation suite is provided as a sample implementation demonstrating approach, structure, and coverage strategy. It is not executed in a live device environment due to setup constraints.

Part 4: Performance & Scale Thinking

1. Load Testing Approach

I would test FroggyTalk using:

- Load testing (normal expected users)
- Stress testing (beyond limits)
- Spike testing (sudden traffic increase)

Focus areas:

- Radio streaming
- Live chat
- Mobile top-up / Pay Utility Bills
- API calls

2. Key Performance Risks

- Streaming lag or buffering
- Chat delays or message loss
- Slow API responses
- Backend failure during payments
- Third-party service delays

3. Tools

- k6 (preferred)
- JMeter (alternative)
- Postman (basic API checks)

4. Success Metrics

- API response time < 2 seconds
- Error rate < 1%
- Stable streaming under load

- Chat messages delivered in real time
- No crashes under peak usage

Part 5: First 30 Days as QA Hire

Phase 1: Understand FroggyTalk Deeply (Days 1–10)

- Make more Manual Testing on FroggyTalk
- Identify critical user journeys
- Review existing bugs and anomalies.

Phase 2: Build QA Foundation (Days 10–20)

- Define QA strategy and test coverage
- Create core test scenarios
- Create and develop basic and initial test automation suite
- Document regression test cases

Phase 3: Release confidence (Days 20–30)

Define a simple Release Readiness Checklist:

- Critical flows (login, payments, top-up) pass on FroggyTalk
- No open high/critical bugs
- Quick smoke test completed

Track basic QA metrics:

- Number of bugs
- Critical issues
- Regression pass/fail

Write effective and powerful Release Notes.